



MILESTONE 3

Silver Layer

Cleaning and conforming Bronze into governed Delta tables with PySpark and MERGE



STEP 1

Create the Databricks Workspace

WHAT

Created an Azure Databricks workspace (dbw-atliq-ab) in rg-atliq-capstone, Central India, Premium tier (Trial with 14 free-DBU days), Hybrid workspace type.

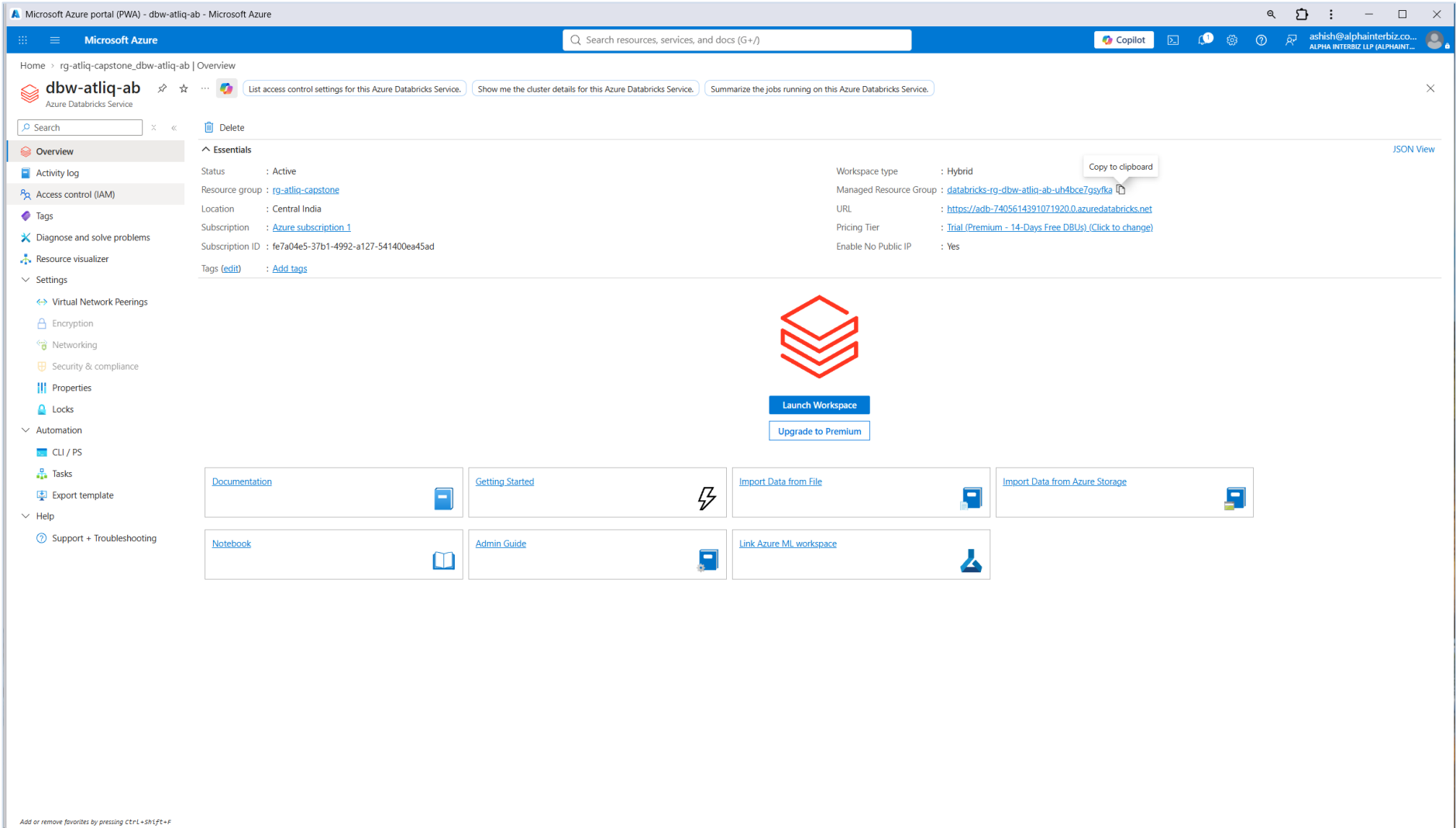
WHY

Premium is required for Unity Catalog governance. Hybrid runs compute on VMs in the subscription and reads the ADLS lake directly, the classic pattern this architecture assumes.

1

STEP 1 · SCREENSHOT

Create the Databricks Workspace



dbw-atliq-ab Overview: Hybrid workspace, Premium trial, Central India



STEP 2

Grant Storage Access

WHAT

Granted the auto-created Access Connector (unity-catalog-access-connector) the Storage Blob Data Contributor role on the ADLS storage account.

WHY

Unity Catalog reaches the lake through this managed identity instead of storage keys in notebooks. One governed identity, granted once, is the secure access pattern, the same secrets hygiene used for Key Vault in M2.

Grant Storage Access

Microsoft Azure portal (PWA) - Access Connector for Azure Databricks - Microsoft Azure

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

ashish@alphainterbiz.co...
ALPHA INTERBIZ LLP (ALPHAINT...

Home

Access Connector for Azure Databricks

Alpha Interbiz LLP (alphainterbiz.com)

Identify non-compliant Access Connectors in my environment. How many Access Connectors do I have in total? List Access Connectors modified in the last 30 days using ARG.

+ Create

Manage view

Refresh

Export to CSV

Open query

Assign tags

Delete

Add to service group

Group by none

Filter for any field...

Subscription equals all

Resource Group equals all

Location equals all

Add filter

☐	Name ↑	Resource Group	Subscription	Subscription ID	Location	State	
☐	unity-catalog-access-connector	...	databricks-rg-dbw-atliq-ab-uh4bce7gsyfka	Azure subscription 1	/subscriptions/fe7a04e5-37b1-4992-a127-54...	Central India	Succeeded

unity-catalog-access-connector, auto-created with the workspace

Grant Storage Access

Microsoft Azure portal (PWA) - atliqlakeab - Microsoft Azure

Home > atliqlakeab

atliqlakeab | Access Control (IAM)

Search resources, services, and docs (G+/)

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Partner solutions

Resource visualizer

Data storage

Containers

Classic file shares

Queues

Tables

Security + networking

Data management

Storage Actions

Redundancy

Data protection

Blob inventory

Static website

Lifecycle management

Settings

Configuration

Resource sharing (CORS)

SFTP

Advisor recommendations

Endpoints

Locks

Check access

Role assignments

Roles

Deny assignments

Number of role assignments for this subscription

6

5000

Search by name or email

Type: All

Role: All

Scope: All scopes

Group by: Role

All (3)

Job function roles (1)

Privileged administrator roles (2)

Name	Type	Role	Scope	Condition
Owner (2)				
Ashish Babaria ashish@alphainterbiz.co...	User	Owner	Subscription (Inherited)	None
Ashish Babaria ashish@alphainterbiz.co...	User	Owner	Subscription (Inherited)	None
Storage Blob Data Contributor (1)				
unity-catalog-access-connector 28a96890-c39e-4f81-ba91-5ea8951b06c1	Managed identity	Storage Blob Data Contributor	This resource	Add

Showing 1 - 3 of 3 results.

atliqlakeab IAM: Storage Blob Data Contributor granted to unity-catalog-access-connector



STEP 3

Wire Unity Catalog to the Lake

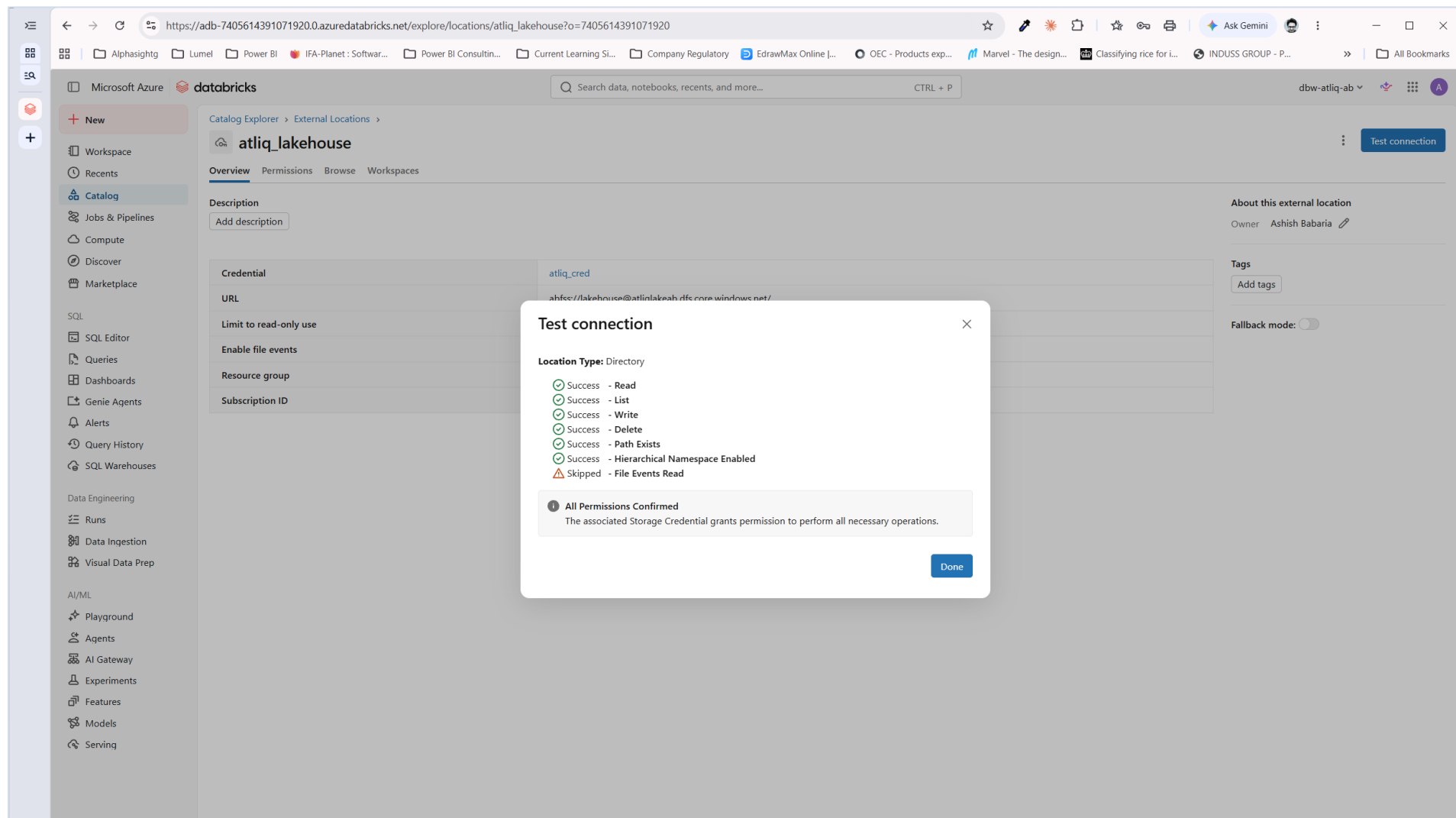
WHAT

Created a Storage Credential (atliq_cred) wrapping the access connector, then an External Location (atliq_lakehouse) mapping it to the lakehouse container. Ran Test connection.

WHY

The storage credential is Unity Catalog's secure handle to ADLS; the external location gives it a governed name. Notebooks reference clean catalog objects instead of raw storage URLs.

Wire Unity Catalog to the Lake



External location atliq_lakehouse: Test connection, All Permissions Confirmed



STEP 4

Create Catalog and Schemas

WHAT

Created the atliq catalog with `MANAGED LOCATION` pointing at the lakehouse container, plus the silver and gold schemas.

WHY

The catalog is the top-level container; silver (cleaned) and gold (star schema) schemas separate the Medallion layers. `MANAGED LOCATION` keeps the catalog's data in the lake, not default metastore storage.

Create Catalog and Schemas

The screenshot shows the Databricks workspace interface. On the left, the sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, Experiments, Features, Models, and Serving. The main area displays a notebook titled '00_uc_setup'. The notebook content shows SQL commands to create a catalog and schemas:

```
%sql
CREATE CATALOG IF NOT EXISTS atliq
MANAGED LOCATION 'abfss://lakehouse@atliqlakeab.dfs.core.windows.net/';

CREATE SCHEMA IF NOT EXISTS atliq.silver;
CREATE SCHEMA IF NOT EXISTS atliq.gold;
```

Below the code, a table shows the execution results of the SQL statements:

Statement	Started At	Tasks	Duration	Rows read	Bytes read	Bytes written
CREATE SCHEMA IF NOT EXISTS atliq.gold	Aug 30, 2026, 01:18 PM	0/0 completed	333 ms	0	0 B	0 B
CREATE SCHEMA IF NOT EXISTS atliq.silver	Aug 30, 2026, 01:18 PM	0/0 completed	986 ms	0	0 B	0 B
CREATE CATALOG IF NOT EXISTS atliq MANAGED LOCATION 'abfss://lakehouse@atliqlakeab.dfs.core.windows.net/';	Aug 30, 2026, 01:18 PM	0/0 completed	1 s 697 ms	0	0 B	0 B

At the bottom of the notebook, a message states: "This result is stored as `_sqlidf` and can be used in other Python and SQL cells."

00_uc_setup: atliq catalog with MANAGED LOCATION, plus silver and gold schemas



STEP 5

Verify Bronze Is Readable

WHAT

Ran a read test from Databricks against bronze/customers, returning 40 rows and displaying the data.

WHY

Confirms the full chain works end to end: Databricks reads the Bronze Parquet that ADF wrote, through the Unity Catalog external location. The hardest wiring, proven before any transformation.

Verify Bronze Is Readable

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

dbw-atliq-ab

File Edit View Run Help Python Last edit was now

Run all Serverless Schedule Comments Share

atliq
MANAGED LOCATION
Expand query text

SQLDF: pyspark.sql.connect.dataframe.DataFrame

OK

This result is stored as `_sqlidf` and can be used in other Python and SQL cells. Send feedback

Bronze read test

1 minute ago (10s) 3 Python

```
df = spark.read.parquet("abfss://lakehouse@atliqlakeab.dfs.core.windows.net/bronze/customers")
print("bronze customers rows:", df.count())
display(df.limit(5))
```

See performance (2)

df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, customer_name: string ... 4 more fields]

bronze customers rows: 40

	customer_id	customer_name	email	city	signup_date	updated_at
1	1	Rahul Verma	rahul.verma1@example.com	Chennai	2025-11-20	2025-11-20T11:00:00.000+00:00
2	2	Krishna Patel	krishna.patel2@example.com	Jaipur	2025-10-11	2025-10-11T09:00:00.000+00:00
3	3	Meera Gupta	meera.gupta3@example.com	Bengaluru	2025-11-16	2025-11-16T09:00:00.000+00:00
4	4	Reyansh Reddy	reyansh.reddy4@example.co...	Surat	2025-03-19	2025-03-19T08:00:00.000+00:00
5	5	Kavya Reddy	kavya.reddy5@example.com	Jaipur	2024-12-01	2024-12-01T14:00:00.000+00:00

5 rows 10.043s runtime Refreshed Now

Bronze read test: bronze customers rows: 40, with sample rows



STEP 6

Build Full-Overwrite Silver

WHAT

Cleaned and loaded the reference and flat-file tables into Silver Delta with overwrite: customers (40), products (25), supplier_price_list (25, latest cost per product), marketing_spend (530).

WHY

Small reference and CSV tables are cheap to reload in full each run, and overwrite is inherently idempotent. Light transforms: trim, standardize case, cast types, dedupe on the business key.

Build Full-Overwrite Silver

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

dbw-atliq-ab

Home

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Catalog

Type to search

My organization

- dbw_atliq_ab
- system
- atliq
 - default
 - gold
 - information_schema
 - silver
- Shares received
 - samples

Silver Layer: Full-Overwrite Reference Tables

Cleans and loads the small reference and flat-file sources into Silver Delta tables. These are reloaded in full each run (overwrite), which is simple and inherently idempotent.

Tables: customers, products, supplier_price_list, marketing_spend

1 minute ago (<1s) 2

```
from pyspark.sql import functions as F, Window

STORAGE_ACCOUNT = "atliqlakeab"
BRONZE = f"abfss://lakehouse@{STORAGE_ACCOUNT}.dfs.core.windows.net/bronze"
```

Customers

Trim names, standardize city casing, cast signup_date, dedupe on customer_id.

Just now (10s) 4

```
customers = (
    spark.read.parquet(f"{BRONZE}/customers")
    .withColumn("customer_name", F.trim("customer_name"))
    .withColumn("city", F.initcap(F.trim("city")))
    .withColumn("signup_date", F.to_date("signup_date"))
    .dropDuplicates(["customer_id"])
    .filter(F.col("customer_id").isNull())
)

customers.write.format("delta").mode("overwrite").option("overwriteSchema", "true") \
    .saveAsTable("atliq.silver.customers")
print("customers ->", customers.count())
```

See performance (2)

customers: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, customer_name: string ... 4 more fields]

customers -> 40

10_silver_full_dims: customers cleaned and overwritten into Silver (40 rows)

6

STEP 6 • SCREENSHOT 2 of 4

Build Full-Overwrite Silver

The screenshot displays the Databricks workspace interface. The left sidebar shows the navigation menu with options like Workspace, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, Experiments, Features, Models, and Serving. The main area shows two notebooks being executed. The first notebook, '10_silver_full_dims', is running a Python script that writes a Delta table 'customers' to the 'silver' schema. The second notebook, 'Products', is running a Python script that reads a Parquet file, trims names, standardizes categories, casts unit prices to decimals, dedupes, and writes a Delta table 'products' to the 'silver' schema. The output of the 'Products' notebook shows 25 rows of data.

```
customers.write.format("delta").mode("overwrite").option("overwriteSchema", "true") \
    .saveAsTable("atliq.silver.customers")
print("customers ->", customers.count())
```

customers: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, customer_name: string ... 4 more fields]
customers -> 40

Products

Trim names, standardize category casing, cast unit_price to decimal, dedupe on product_id.

```
products = (
    spark.read.parquet(f"{BRONZE}/products")
    .withColumn("product_name", F.trim("product_name"))
    .withColumn("category", F.initcap(F.trim("category")))
    .withColumn("unit_price", F.col("unit_price").cast("decimal(10,2)"))
    .dropDuplicates(["product_id"])
    .filter(F.col("product_id").isNull())
)
products.write.format("delta").mode("overwrite").option("overwriteSchema", "true") \
    .saveAsTable("atliq.silver.products")
print("products ->", products.count())
```

products: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, product_name: string ... 3 more fields]
products -> 25

Products: trimmed, category standardized, unit_price cast to decimal (25 rows)

Build Full-Overwrite Silver

The screenshot displays the Databricks web interface. On the left, a sidebar contains navigation links for Workspace, Catalog, and various tools like SQL Editor and Dashboards. The main area shows a notebook titled "Supplier Price List (from CSV)". Below the title, a text box explains the goal: "Keep only the latest effective_date per product using a window function, so each product has one current supplier cost." The notebook code is as follows:

```
spl_raw = spark.read.parquet(f"{BRONZE}/supplier_price_list")
w = Window.partitionBy("product_id").orderBy(F.col("effective_date").desc())
supplier = (
    spl_raw
    .withColumn("supplier_cost", F.col("supplier_cost").cast("decimal(10,2)"))
    .withColumn("effective_date", F.to_date("effective_date"))
    .withColumn("rn", F.row_number().over(w).filter("rn = 1").drop("rn"))
    .filter(F.col("product_id").isNotNull())
)
supplier.write.format("delta").mode("overwrite").option("overwriteSchema", "true") \
    .saveAsTable("atliq.silver.supplier_price_list")
print("supplier_price_list ->", supplier.count())
```

Below the code, the execution results are shown: "2 dataframes: spl_raw, supplier" and "supplier_price_list -> 25".

Supplier price list: latest effective_date per product kept (25 rows)

Build Full-Overwrite Silver

The screenshot displays the Databricks interface with a Python notebook titled "Marketing Spend (from CSV)". The notebook's content is as follows:

```
File Edit View Run Help Python Last edit was now

Marketing Spend (from CSV)
Cast date and numeric columns, full overwrite.

mkt = (
    spark.read.parquet(f"{BRONZE}/marketing_spend")
    .withColumn("spend_date", F.to_date("spend_date"))
    .withColumn("spend_amount", F.col("spend_amount").cast("decimal(12,2)"))
    .withColumn("clicks", F.col("clicks").cast("int"))
)

mkt.write.format("delta").mode("overwrite").option("overwriteSchema", "true") \
    .saveAsTable("atliq.silver.marketing_spend")
print("marketing_spend ->", mkt.count())

> See performance (2)
```

The output of the notebook shows the schema of the resulting DataFrame:

```
mkt: pyspark.sql.connect.dataframe.DataFrame
  spend_date: date
  channel: string
  campaign: string
  spend_amount: decimal(12,2)
  clicks: integer

marketing_spend -> 530
```

Marketing spend: dates and numerics cast, full overwrite (530 rows)



STEP 7

Build Incremental MERGE Silver

WHAT

Loaded the transactional tables via Delta MERGE on the business key: orders (300) and payments (246) update-when-newer, order_items (783) insert-only.

WHY

MERGE on the business key is what makes Silver idempotent: re-running updates matching rows in place instead of appending duplicates. order_items is insert-only because a line item never changes once written.

Build Incremental MERGE Silver

The screenshot displays the Databricks interface with a notebook titled "20_silver_incremental_merge". The notebook content includes a Python code block that reads a table named "orders", deduplicates it by order_id, and merges it into a target table. The output shows the "orders silver count" as 300.

```
python
w_ord = Window.partitionBy("order_id").orderBy(F.col("updated_at").desc())
orders = (
    read_all("orders")
    .withColumn("rn", F.row_number().over(w_ord).filter("rn = 1").drop("rn"))
    .withColumn("order_date", F.to_date("order_date"))
    .withColumn("status", F.initcap(F.trim("status")))
    .withColumn("order_amount", F.col("order_amount").cast("decimal(12,2)"))
    .filter(F.col("order_id").isNotNull())
)
(DeltaTable.createIfNotExists(spark)
 .tableName("atliq.silver.orders").addColumns(orders.schema).execute())
(DeltaTable.forName(spark, "atliq.silver.orders").alias("t")
 .merge(orders.alias("s"), "t.order_id = s.order_id")
 .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
 .whenNotMatchedInsertAll()
 .execute())
print("orders silver count ->", spark.table("atliq.silver.orders").count())
```

orders: pyspark.sql.connect.dataframe.DataFrame = [order_id: integer, customer_id: integer ... 5 more fields]
orders silver count -> 300

20_silver_incremental_merge: orders deduplicated and MERGED (300 rows)

Build Incremental MERGE Silver

The screenshot displays the Databricks interface with a notebook titled "20_silver_incremental_merge". The notebook content is as follows:

```
File Edit View Run Help Python Last edit was now
orders silver count -> 300

Payments
Same upsert pattern on payment_id: update when newer, else insert.

w_pay = Window.partitionBy("payment_id").orderBy(F.col("updated_at").desc())
payments = (
    read_all("payments")
    .withColumn("rn", F.row_number().over(w_pay).filter("rn = 1").drop("rn"))
    .withColumn("amount", F.col("amount").cast("decimal(12,2)"))
    .withColumn("paid_at", F.col("paid_at").cast("timestamp"))
    .filter(F.col("payment_id").isNotNull())
)
(DeltaTable.createIfNotExists(spark)
 .tableName("atliq.silver.payments").addColumns(payments.schema).execute())
(DeltaTable.forName(spark, "atliq.silver.payments").alias("t")
 .merge(payments.alias("s"), "t.payment_id = s.payment_id")
 .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
 .whenNotMatchedInsertAll()
 .execute())
print("payments silver count ->", spark.table("atliq.silver.payments").count())
```

The output of the notebook execution is:

```
> See performance (3)
> payments: pyspark.sql.connect.dataframe.DataFrame = [payment_id: integer, order_id: integer ... 4 more fields]
payments silver count -> 246
```

Payments: same upsert pattern on payment_id (246 rows)

Build Incremental MERGE Silver

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Workspace, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, Experiments, Features, Models, and Serving. The main area displays a notebook titled '20_silver_incremental_merge'. The notebook content includes a section titled 'Order Items' with the text: 'Insert-only: a line item is immutable once created, so we only insert new order_item_ids, never update.' Below this is a code block with the following Python code:

```
w_it = Window.partitionBy("order_item_id").orderBy(F.col("created_at").desc())
order_items = (
    read_all("order_items")
    .withColumn("rn", F.row_number().over(w_it).filter("rn = 1").drop("rn"))
    .withColumn("quantity", F.col("quantity").cast("int"))
    .withColumn("item_price", F.col("item_price").cast("decimal(10,2)"))
    .filter(F.col("order_item_id").isNotNull())
)
(DeltaTable.createIfNotExists(spark)
 .tableName("atliq.silver.order_items").addColumns(order_items.schema).execute())
(DeltaTable.forName(spark, "atliq.silver.order_items").alias("t")
 .merge(order_items.alias("s"), "t.order_item_id = s.order_item_id")
 .whenNotMatchedInsertAll()
 .execute())
print("order_items silver count ->", spark.table("atliq.silver.order_items").count())
```

Below the code block, the output shows: 'order_items: pyspark.sql.connect.dataframe.DataFrame = [order_item_id: integer, order_id: integer ... 4 more fields]' and 'order_items silver count -> 783'.

Order items: insert-only MERGE on order_item_id (783 rows)

Bronze to Silver: What Was Transformed

Silver is not a copy of Bronze. Its job is to clean, type, deduplicate, and conform, one correct row per entity.

Cleaning

- Trim whitespace on text fields (names, status)
- Standardize casing with initcap (city, category, status)
- Cast to correct types: prices to decimal, dates to date, quantity and clicks to int

Deduplication

- dropDuplicates on the business key (customer_id, product_id)
- Window row_number keeps the latest row per key before MERGE
- Filter out rows with a null business key

Business logic

- supplier_price_list: keep only the latest effective_date per product (one current cost)
- Drop the ingest_date partition column so Bronze physical detail does not leak into Silver

Load strategy

- Reference and CSV tables: full overwrite (latest snapshot)
- orders, payments: MERGE upsert, update when newer
- order_items: insert-only MERGE (line items are immutable)



STEP 8

Prove MERGE Was Used

WHAT

Ran DESCRIBE HISTORY atliq.silver.orders, which records a MERGE operation with its metrics (rows inserted / updated).

WHY

This is the direct evidence that upsert logic drives the load, not a naive overwrite. The Delta transaction log is the audit trail of exactly how each table was written.

Prove MERGE Was Used

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, Experiments, Features, Models, and Serving.

The main area displays a SQL query result for the command: `%sql DESCRIBE HISTORY atliq.silver.orders`. The result is a table with 7 columns: version, timestamp, userid, username, operation, and operationParameters. The table shows two rows of history.

version	timestamp	userid	username	operation	operationParameters
1	2026-08-30T08:33:48.000+00:00	144749938958965	ashish@alphainterbiz.co...	MERGE	[{"predicate":["(order_id#13734 = order_id#13679)\\"]","clusterBy":[""]}]
2	2026-08-30T08:33:45.000+00:00	144749938958965	ashish@alphainterbiz.co...	CREATE TABLE	[{"partitionBy":[""],"clusterBy":[""],"description":null,"isManaged":true}]

The table shows 2 rows and 4.965s runtime. The result is stored as `_sqlidf` and can be used in other Python and SQL cells.

DESCRIBE HISTORY atliq.silver.orders: CREATE TABLE, then MERGE



STEP 9

Reconcile the Silver Layer

WHAT

Ran a reconciliation query: $\text{SUM}(\text{order_amount})$ from orders equals $\text{SUM}(\text{quantity} \times \text{item_price})$ from order_items (both ~2,076,041).

WHY

Matching the M1 reconciliation number proves the data survived the Bronze to Silver transformation intact, nothing dropped, duplicated, or corrupted along the way.

Reconcile the Silver Layer

The screenshot displays the Databricks interface with a SQL query executed in the workspace. The query is titled "reconciliation in Silver" and is stored in a variable named `_sqldf`. The query calculates the sum of order amounts, line item totals, and payment totals from the `atliq.silver` schema.

```
%sql
SELECT
  (SELECT SUM(order_amount) FROM atliq.silver.orders) AS header_total,
  (SELECT SUM(quantity * item_price) FROM atliq.silver.order_items) AS line_item_total,
  (SELECT SUM(amount) FROM atliq.silver.payments) AS payment_total
```

The result is displayed as a table with 1 row and 3 columns: `.00 header_total`, `.00 line_item_total`, and `.00 payment_total`. The values for `header_total` and `line_item_total` are both 2076041.00.

	.00 header_total	.00 line_item_total	.00 payment_total
1	2076041.00	2076041.00	1745772.00

The interface also shows the Databricks catalog on the left, the SQL Editor on the right, and a notification that the result is stored in `_sqldf` and can be used in other Python and SQL cells.

Silver reconciliation: $header_total = line_item_total = 2,076,041.00$

10

STEP 10

Prove Silver Idempotency

WHAT

Re-ran the MERGE notebook. Row counts stayed identical (300 / 246 / 783), and DESCRIBE HISTORY recorded a second MERGE with 0 rows inserted (all updated in place).

WHY

The counts not moving on a re-run is the proof: MERGE recognizes existing rows and updates them rather than duplicating. Silver is safe to re-run, the transformation-layer equivalent of M2's ingestion idempotency.

Prove Silver Idempotency

The screenshot shows a Databricks notebook titled "Orders" with the following content:

Orders

Deduplicate to the latest row per order_id, then MERGE. Update only when the incoming row is newer (updated_at), else insert.

```
02:16 PM (6s) 5
w_ord = Window.partitionBy("order_id").orderBy(F.col("updated_at").desc())
orders = (
  read_all("orders")
  .withColumn("rn", F.row_number().over(w_ord).filter("rn = 1").drop("rn"))
  .withColumn("order_date", F.to_date("order_date"))
  .withColumn("status", F.initcap(F.trim("status")))
  .withColumn("order_amount", F.col("order_amount").cast("decimal(12,2)"))
  .filter(F.col("order_id").isNotNull())
)
(DeltaTable.createIfNotExists(spark)
 .tableName("atliq.silver.orders").addColumns(orders.schema).execute())
(DeltaTable.forName(spark, "atliq.silver.orders").alias("t")
 .merge(orders.alias("s"), "t.order_id = s.order_id")
 .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
 .whenNotMatchedInsertAll()
 .execute())
print("orders silver count ->", spark.table("atliq.silver.orders").count())
> See performance (3)
> orders: pyspark.sql.connect.dataframe.DataFrame = [order_id: integer, customer_id: integer ... 5 more fields]
orders silver count -> 300
```

Payments

Same upsert pattern on payment_id: update when newer, else insert.

```
02:16 PM (5s) 7
w_pay = Window.partitionBy("payment_id").orderBy(F.col("updated_at").desc())
payments = (
  read_all("payments")
  .withColumn("rn", F.row_number().over(w_pay).filter("rn = 1").drop("rn"))
```

Rerun: orders silver count still 300

Prove Silver Idempotency

The screenshot shows the Databricks web interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, Experiments, Features, Models, and Serving. The main area displays a notebook titled 'Payments' with the following content:

Payments

Same upsert pattern on payment_id: update when newer, else insert.

```
02:16 PM (5s) 7
w_pay = Window.partitionBy("payment_id").orderBy(F.col("updated_at").desc())
payments = (
    read_all("payments")
    .withColumn("rn", F.row_number().over(w_pay)).filter("rn = 1").drop("rn")
    .withColumn("amount", F.col("amount").cast("decimal(12,2)"))
    .withColumn("paid_at", F.col("paid_at").cast("timestamp"))
    .filter(F.col("payment_id").isNull())
)
(DeltaTable.createIfNotExists(spark)
 .tableName("atliq.silver.payments").addColumns(payments.schema).execute())
(DeltaTable.forName(spark, "atliq.silver.payments").alias("t")
 .merge(payments.alias("s"), "t.payment_id = s.payment_id")
 .whenMatchedUpdateAll(condition="s.updated_at > t.updated_at")
 .whenNotMatchedInsertAll()
 .execute())
print("payments silver count ->", spark.table("atliq.silver.payments").count())
> See performance \(3\)
```

payments: pyspark.sql.connect.dataframe.DataFrame
payment_id: integer
order_id: integer
amount: decimal(12,2)
method: string
paid_at: timestamp
updated_at: timestamp

payments silver count -> 246

Order Items

Insert-only: a line item is immutable once created, so we only insert new order_item_ids, never update.

```
02:16 PM (3s) 9
w_it = Window.partitionBy("order_item_id").orderBy(F.col("created_at").desc())
```

Rerun: payments silver count still 246

Prove Silver Idempotency

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, Experiments, Features, Models, and Serving.

The main area displays a notebook titled "Order Items". The notebook content includes a description: "Insert-only: a line item is immutable once created, so we only insert new order_item_ids, never update." Below this is a code cell with a Scala script that reads all order items, filters for new items (rn = 1), and inserts them into the atliq.silver.order_items table using DeltaTable.createIfNotExists and DeltaTable.merge. The script also prints the count of order items in the silver table.

Below the code cell, the query history table is shown, displaying the execution of the %sql DESCRIBE HISTORY atliq.silver.orders command. The table has columns: Statement, Started At, Tasks, Duration, Rows read, Bytes read, and Bytes written. The execution status is "8/8 completed" with a duration of 537 ms.

Statement	Started At	Tasks	Duration	Rows read	Bytes read	Bytes written
%sql DESCRIBE HISTORY atliq.silver.orders	Aug 30, 2026, 02:16 PM	8/8 completed	537 ms	0	0 B	0 B

Rerun: order_items silver count still 783

Prove Silver Idempotency

The screenshot shows the Databricks workspace interface. The left sidebar contains navigation options like Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, Agents, AI Gateway, Experiments, Features, Models, and Serving.

The main area displays a notebook titled "MERGE proof" with a SQL cell showing the command `%sql DESCRIBE HISTORY atliq.silver.orders`. The output shows a table with 3 rows and 7 columns: version, timestamp, userid, userName, operation, and operationParameters. The table shows two MERGE operations and one CREATE TABLE operation.

Below the "MERGE proof" notebook, there is a notebook titled "reconciliation in Silver" with a SQL cell showing a complex query using `SELECT` and `FROM` clauses to calculate totals from different tables. The output shows a DataFrame with columns: header_total, line_item_total, and payment_total.

version	timestamp	userid	userName	operation	operationParameters
1	2026-08-30T08:46:50.000+00:00	144749938958965	ashish@alphainterbiz.co...	MERGE	> [{"predicate":"(order_id#15477 = order_id#15429)"}],{"clusterBy":""}
2	2026-08-30T08:33:48.000+00:00	144749938958965	ashish@alphainterbiz.co...	MERGE	> [{"predicate":"(order_id#13734 = order_id#13679)"}],{"clusterBy":""}
3	2026-08-30T08:33:45.000+00:00	144749938958965	ashish@alphainterbiz.co...	CREATE TABLE	> [{"partitionBy":"","clusterBy":"","description":null,"isManaged":true}

header_total	line_item_total	payment_total
00	00	00

After the rerun: DESCRIBE HISTORY shows a second MERGE version, reconciliation unchanged

Milestone 3 Complete

What this milestone delivered

Governed Silver layer

7 clean Delta tables in Unity Catalog, reached through a managed-identity external location, no keys in code.

MERGE upserts

Transactional tables loaded by Delta MERGE on the business key. Proven with DESCRIBE HISTORY.

Proven idempotent

Re-running left counts at 300 / 246 / 783. The transformation is safe to re-run, no duplication.

Verified Silver load

40

customers

25

products

300

orders

783

order items

246

payments

Real issues handled: requested a vCPU quota increase (subscription started at 0 cores); regenerated Bronze after an M2 same-day idempotency re-run had emptied a date-partitioned folder.